

gen4-uLCD-35DCT
gen4-uLCD-35DCT-CLB
gen4-uLCD-43D Series
gen4-uLCD-43D Series
gen4-uLCD-43D-CLB-HR
gen4-uLCD-43D-CLB
gen4-uLCD-43D-HR
gen4-uLCD-43D
gen4-uLCD-43DCT-CLB-HR
gen4-uLCD-43DCT-CLB
gen4-uLCD-43DCT-HR
gen4-uLCD-43DCT
gen4-uLCD-43DT-HR
gen4-uLCD-43DT
gen4-uLCD-50D Series
gen4-uLCD-50D Series
gen4-uLCD-50D-CLB
gen4-uLCD-50D
gen4-uLCD-50DCT
gen4-uLCD-50DCT-CLB
gen4-uLCD-50DT
gen4-uLCD-70D Series
gen4-uLCD-70D Series
gen4-uLCD-70D-CLB
gen4-uLCD-70D
gen4-uLCD-70DCT
gen4-uLCD-70DCT-CLB
gen4-uLCD-70DT
gen4-uLCD Pixxi44 Series
gen4-uLCD Pixxi44 Series
gen4-uLCD-24P4 Series
gen4-uLCD-24P4 Series
gen4-uLCD-24P4-CLB
gen4-uLCD-24P4
gen4-uLCD-24P4T
gen4-uLCD-24P4CT
gen4-uLCD-24P4CT-CLB
gen4-uLCD-28P4 Series
gen4-uLCD-28P4 Series
gen4-uLCD-28P4-CLB
gen4-uLCD-28P4
gen4-uLCD-28P4T
gen4-uLCD-28P4CT
gen4-uLCD-28P4CT-CLB
gen4-uLCD-32P4 Series
gen4-uLCD-32P4 Series
gen4-uLCD-32P4-CLB
gen4-uLCD-32P4
gen4-uLCD-32P4T
gen4-uLCD-32P4CT
gen4-uLCD-32P4CT-CLB
gen4-uLCD-35P4 Series
gen4-uLCD-35P4 Series
gen4-uLCD-35P4-CLB
gen4-uLCD-35P4
gen4-uLCD-35P4T
gen4-uLCD-35P4CT
gen4-uLCD-35P4CT-CLB
gen4-uLCD-43P4 Series
gen4-uLCD-43P4 Series
gen4-uLCD-43P4-CLB-HR
gen4-uLCD-43P4-CLB

4DGL Programmer's Reference

Introduction

The 4D-Labs family of embedded graphics processors (GOLDELOX, PICASO, PIXXI-28, PIXXI-44 and DIABLO-16) are powered by a highly optimized soft-core virtual engine, E.V.E. (Extensible Virtual Engine).

EVE is a proprietary, high performance virtual processor with an extensive byte-code instruction set optimized to execute compiled 4DGL programs. 4DGL (4D Graphics Language) was specifically developed from ground up for the EVE engine core. It is a high-level language which is easy to learn and simple to understand yet powerful enough to tackle many embedded graphics applications.

4DGL is a graphics-oriented language allowing rapid application development. An extensive library of graphics, text and file system functions and the ease of use of a language that combines the best elements and syntax structure of languages such as C, Basic, Pascal, etc. Programmers familiar with these languages will feel right at home with 4DGL. It includes many familiar instructions such as `IF..ELSE..ENDIF`, `WHILE..WEND`, `REPEAT..UNTIL`, `GOSUB..ENDSUB`, `GOTO` as well as a wealth of (chip-resident) internal functions that include `SERIN`, `SEROUT`, `GFX_LINE`, `GFX_CIRCLE` and many more.

This document covers the language style, the syntax and flow control. This document should be used in conjunction with processor specific internal functions manuals.

- [PIXXI Internal Functions Manual](#)
- [DIABLO-16 Internal Functions Manual](#)
- [PICASO Internal Functions Manual](#)
- [GOLDELOX Internal Functions Manual](#)

Language Summary

This document is made up of the following sections:

- [Language Style](#)
 - Numbers, identifiers, comments
- [Constants and Variables](#)
 - `var`, `var private`, `#constant`, Inbuilt Constants, `#CONST...#END`, `#DATA...#END`
- [Pre-Processor Directives](#)
 - `#IF`, `#IFNOT`, `#ELSE`, `#ENDIF`, `EXISTS`, `#ERROR`, `#MESSAGE`, `#NOTICE`, `sizeof`, `argcount`, `#STOP`, `#USE .. USING`, `#inherit`, `#MODE`
- [Expressions and Operators](#)
 - `:=`, `&(address modifier)`, `,`, `+`, `-`, `,`, `/`, `%`, `&(as a logical operator)`, `!`, `^`, `==`, `!=`, `>`, `<`, `<=`, `&&`, `||`, `!`, `~`, `<<`, `>>`, `++`, `-`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, ternary operators
- [Language Flow Control](#)
 - `if .. else .. endif`, `while .. wend`, `repeat .. until/forever`, `goto`, `for .. next`, `switch .. case`
- [Functions and Subroutines](#)
 - `gosub .. endsub`, `func .. endfunc`, `return`, `SystemReset`, `ProgramExit`, `Argcount`, `@(argument pointer)`

Language Style

4DGL is a case sensitive language. The colour of the text, in the Workshop4 IDE, reveals if the syntax will be accepted by the compiler.

Numbers

Numbers may be defined as decimal, hex or binary.

```
0xAA55      // hex number
-1234       // decimal number
0b10011001  // binary number
```

Identifiers

Identifiers are names used for referencing variables, constants, functions, and subroutines.

A valid identifier:

- Must begin with a letter of the English alphabet or possibly the underscore (`_`)
- Consists of alphanumeric characters and the underscore (`_`)
- May not contain special characters: `~ ! @ # $ % ^ & * ( ) ` - = { } [] : " ; ' < > ? , . / |`

On This Page
Introduction
Language Summary
Language Style
Numbers
Identifiers
Comments
Redefining Pre-Processor Directives
Constants and Variables
Constants
Inbuilt Constants
Variables
Private Variables
Data Blocks
Pre-Processor Directives
Conditional Pre-Processor Directives
<code>#STOP</code>
<code>#USE</code> and <code>#IF USING</code>
<code>#MODE</code>
<code>#STACK</code>
<code>#inherit</code>
Expression and Operators
Assignment Operator
Address Modifier
<code>[&]</code> Get Variable Address
<code>[*]</code> Use Variable as Pointer
Arithmetic Operators
<code>[+]</code> Addition
<code>[-]</code> Subtraction
<code>[*]</code> Multiplication
<code>[/]</code> Division
<code>[%]</code> Modulus
Comparison Operators
<code>[==]</code> Equality
<code>[!=]</code> Inequality
<code>[>]</code> Greater Than
<code>[>=]</code> Greater Than or Equals
<code>[<]</code> Less Than
<code>[<=]</code> Less Than or Equals
Logical Operators

Elements ignored by the compiler include spaces, new lines, and tabs. All these elements are collectively known as the **white space**. White space serves only to make the code more legible. It does not affect the actual compiling.

Note that an identifier for a subroutine must have a colon (:) appended to it. Subroutine names are scoped locally inside functions, they are not accessible outside the function.

```
mysub:
    [statements]
endsub;
```

Comments

A comment is a line or paragraph of text in a program file such that that line or paragraph is not considered when the compiler is processing the code of the file.

To write a comment on one line, type two forward slashes `//` and type the comment. Anything on the right side of both forward slashes will not be read by the compiler.

Also you can comment in multiple lines by enclosing between `/*` and `*/`. See example.

Examples

Single line Comment

// This is my comment

Single line Block Comment

/* This is my one line comment */

Multi-Line Block Comment

/*
 * This is a multi-line comment
 * which can span over many lines
 */

It is good to comment your code so that you or anybody else can later understand it. Also, comments are useful to comment out sections of the code, so it can be left as a reminder about a modification that was made and perhaps be modified or repaired later. Comments should give meaningful information on what the program is doing. Comment such as ‘Set output 4’ fails to state the purpose of the instruction. Something like ‘Turn Talk LED ON’ is much more useful.

Redefining Pre-Processor Directives

It is possible to add your own ‘flavour’ to the pre-processor by using the `$` (substitution directive).

For example, if you wanted to make your code look ‘more C like’ you can do the following,

Example:

```
// Define some user preferences to make things look more 'C' like
#constant enum      $#constant      /* define the enum word ... */
#constant #define   $#constant      /* define the #define word ... */
#constant #ifdef     $IF EXISTS      /* define the #ifdef word ... */
#constant #ifndef    $IFNOT EXISTS   /* define the #ifndef word ... */
#constant #else      $ELSE           /* define the #else word ... */
#constant #endif     $ENDIF          /* define the #endif word ... */
#constant #include    $inherit       /* define the #include word */
#constant #if         $IF            /* define the #if word ... */
```

Now, the compiler will use these as aliases to the default directives.

Constants and Variables

Constants

A constant is a data value that cannot be changed during run-time. A constant can be declared as a single line entry with the `#constant` directive. A block of constants can be declared with the `#CONST` and `#END` directives. Every constant is declared with a unique name which must be a valid identifier.

It is a good practice to write constant names in uppercase. The constant’s value can be expressed as decimal, binary, hex or string. If a constant’s value is prepended with a `$` it becomes a complete text substitution with no validation during pre-processing.

Syntax:

Simple

#constant NAME value

Multiple

#constant NAME1 := value1, NAME2 := value2,..., NAMEN := valueN

Equation

#constant NAME equation

Text Substitution

#constant NAME \$TextSubstitution

Block of Constants

#CONST
 BUTTONCOLOUR 0xC0C0
 SLIDERMAX 200
#END

	Description
#constant	The required symbol to define
name	The name of the constant
value	A value to set the symbol to

Inbuilt Constants

The compiler now checks the program and memory sizes and reports error if amount is exceeded. This used to be done by Workshop4. The following constants must be defined in the platforms *fnc* file for this to work.

Example:

```
#constant __MAXMEM      255
#constant __MAXPROG 9216
```

To allow the programmer to recognise what platform is being used, the device is also described with a constant in the *fnc* file.

For example, in GOLDELOX displays, we have:

```
#constant GOLDELOX1
#constant __PLATFORM GOLDELOX
```

This allows the programmer to make coding decisions:

```
#IF __PLATFORM == GOLDELOX
// code it this way
#ELSE
// code it that way
#ENDIF
```

Variables

Like most programming languages, **4DGL** can use and process named variables and their contents. Variables are simply names used to refer to some location in memory - a location that holds a value with which we are working with. Variables used by the -GFX based target platforms are signed 16-bit. Variables are defined with the **var** statement, and are visible globally if placed outside a function (scope is global) or private if placed inside a function (scope is local).

Note

A **var** is a 16-bit signed integer with a range of -32768 to 32767

Example

```
var ball_x, ball_y, ball_r;
var ball_colour;
var xdir, var ydir;
```

Variables can also be an array such as:

```
var PlotInfoX[100], PlotInfoY[100];  // Index starting from 0 to 99.
```

Global and local variables can be initialised when they are declared and are initialised to 0 by default. Local arrays are supported; however, they must be used with caution due to stack size limitations, especially on GOLDELOX. A variable or array can only be initialised with a constant value.

Example

```
var myvar;                  // initialise to 0 when declared
var myvar2 := 100;          // initialise to 100 when declared
var myArray[4] := [5, 200, 500, 2000];  // initialise the array
```

You can also size an array by setting certain initialization values,

```
var myArray[] := [1, 2, 3, 4];      // array is sized to 4
```

And, partial initializing,

```
var myBuffer[10] := [0xAA, 0x55];  // create buffer, initialize 2 entries
```

Note that arrays can only have a single dimension. Variables can also hold a pointer to a function or pointer to other variables including arrays. Also, the index starts from 0.

Examples

Store Characters

```
var buffer1[30]; // general purpose buffer for up to 60 bytes
var buffer2[30]; // general purpose buffer for up to 60 bytes

func main()
    buffer1[0] := 'AB'; // put some characters into the buffer
    buffer1[1] := 'CD';
    buffer2[0] := 'EF';
    // buffers are 16bits per location so chars are packed

    buffer2[1] := 0;
    // ...
endfunc
```

Function Pointers

```
var funclist[2];

func foo()
    // [...some code here...]
endfunc

func baa()
    // [...some code here...]
endfunc

func main()
    // load the function pointers into an array
    funclist[0] := foo;
    funclist[1] := baa;

    funclist[0](); // execute foo
    funclist[1](); // execute baa
endfunc
```

Notes concerning variables

- Global variables and arrays are persistent and exist during the entire execution of a program.
- Global variables and arrays are visible to all functions and are considered a shared resource.
- Local variables are created on the stack and are only visible inside a function call.
- Local variables are released at the end of a function call.
- More data types will be added in future releases.

Private Variables

In the normal course of program execution, any variables declared locally in functions are discarded when the function exits, freeing up the stack.

If we want the value of a variable or variable array to be retained after the function has executed, we simply declare the variable as private.

A private variable may be initialised just like a normal global or local variable, however, it will only be initialised once during the program initialisation in the start-up code (just like a global variable); therefore, each time the function is called, the value that was set during the last call - is persistent, OR it is the value that was set during initialisation (if this is the first call, or it has not been changed).

Example

```
func myfunc()
var private hitcounter := 100; // initial hitcounter value is 100
print("\nHits = ",hit"ounter++")
endfunc
```

Data Blocks

#DATA blocks reside in the CODE space and can be **bytes** or **words**. Data cannot be changed during run-time (i.e., it is read only). A block of data can be indexed like an array. A block of data is declared with the **#DATA** and **#END** directives. Every data entry is declared with a unique name which must be a valid identifier.

Syntax

Inline

```
#DATA
    type name    value1, value2, /* ... */ valueN
#END
```

Multi-Line

```
#DATA
    type name
    value1, value2, /* ... */ valueN
#END
```

Equation

```
#DATA
    type name    equation1, equation2, /* ... */ equationN
#END
```

Description	
#DATA	The required symbol to define
type	byte or word data type keyword
name	The name of the data array

Description	
value	A list of 8 bit or 16 bit values in the data array

Examples

Simple

```
#DATA
    word values      0x0123, 0x4567, 0x89AB, 0xCDEF
    byte hexval      "0123456789ABCDEF"
#END

// and in a function, ...
func main()
    var ch, wd, index1, index2;
    ch := hexval[index1];

    // load ch with the correct ascii character

    wd := values[index2];    // get the required value to wd
    // ...
endfunc
```

Pre-computed Values

```
#constant SPEED 33

#DATA
    word mylimits (BUFSIZE1+BUFSIZE2)*750/1000, 100, 200, (SPEED*20)
#END

func main()
    var n;
    for (n:=0; n < sizeof(mylimits); n++) print(mylimits[n], "\n");
endfunc
```

Function Pointers

```
#DATA
    word myfuncs    tom, joey, harry
#END

func main()
    var n;
    for (n:=0; n < sizeof(myfuncs); n++) myfuncs[n]();
endfunc

func tom()
    putstr("Tom\n");
endfunc

func joey()
    putstr("Joey\n");
endfunc

func harry()
    putstr("Harry\n");
endfunc
```

Pre-Processor Directives

Conditional Pre-Processor Directives

These pre-processor directives allow you to test the value of an arithmetic expression, or the existence of a pre-defined constant or PmmC function name, or the size of a predefined array.

- #IF
- #IFNOT
- #IF EXISTS
- #IFNOT EXISTS
- #IF USING
- #ELSE
- #ENDIF

#IFNOT gives the inverse response of #IF. EXISTS can be appended to these two to test the existence of a pre-defined constant or PmmC function name. USING can be appended to #IF to determine whether or not a certain function is to be used and therefore, compiled.

Combined with #ELSE and #ENDIF, conditional inheritance of other files and control of blocks of code can be performed.

The following pre-processor directives can be used together with #IF and #IFNOT for specific evaluation:

- sizeof - size of a predefined array
- argcount - number of arguments required by User/PmmC function

The following pre-processor directives can be use to provide helpful notices, warnings and showing errors.

- #MESSAGE
- #NOTICE
- #ERROR

#STOP

This pre-processor directive terminates compilation, it is mainly used for debugging purposes to disable compilation of code from a certain point onwards.

Example

```
#STOP // compiler will stop here
```

#USE and #IF USING

The compiler will compile all functions it finds but will only link functions that are referenced. This allows to inherit other files that could be considered as function libraries with your favourite routines in them.

To reduce compile time and improve code readability, the #USE and USING pre-processor directives are used to selectively include functions in inherited files, allowing you to build libraries of code, but only selectively inherit the functions required by the project.

Library File

```
#IF USING tom
func tom()
    putstr("Tom\n");
endfunc
#endif

#IF USING joey
func joey()
    putstr("Joey\n");
endfunc
#endif

#IF USING harry
func harry()
    putstr("Harry\n");
endfunc
#endif

#IF USING george
func george()
    putstr("George\n");
endfunc
#endif
```

Main File

```
#USE tom, george

// this instructs the compiler to compile those
// functions and use them in your program

func main()
    tom();
    george();
endfunc
// note that joey and harry are not compiled or linked
```

#MODE

This is used to define pre-processor directives such as the memory section you want you program to run from, and Flash Memory protection. These pre-processor Directives do not apply to GOLDELOX.

#MODE is defined at the start of your program, typically after any #inherit lines.

PICASO, PIXXI-28 and PIXXI-44 Processors have the following directives available.

Directive	Description
RUNFLASHH	Program will execute from processor Flash Memory. If omitted, the Program loaded in Flash memory will load to RAM before it is run. A program that runs directly from Flash will run a little slower than a program that is run from RAM but has the advantage of leaving RAM space free for other functions.

DIABLO-16 has the following directives available.

Directive	Description
RUNFLASH	Irrelevant for DIABLO as the RUNFLASH overhead for DIABLO is negligible and therefore is always used. The Program will execute from processor Flash Memory whether this is defined or not. Listed for information purposes only.
FLASHBANK_0 FLASHBANK_1 FLASHBANK_2 FLASHBANK_3 FLASHBANK_4 FLASHBANK_5	Program target is Flashbank 0 (or 1, 2, 3, 4, 5 as defined). This is useful when writing child programs and defining which Flashbank the child program will reside in.
SAVE_TO_DISK	This saves the current program to the uSD on the display. Ensure that the program file on your computer is saved and has an 8.3 filename, otherwise the first 12 characters will become the filename.ext
FLASH_READ_PROTECT	This makes the Flashbank that the code is written to read protected so that it cannot be read by flash read type commands.

Directive	Description
FLASH_WRITE _PROTECT	This makes the Flashbank that the code is written to write protected so it cannot be overwritten using the various flash write type commands. It can only be erased using <code>flash_EraseBank()</code> with the confirmation.

If multiple `#MODE` directives are required together, such as Flashbank with Read and/or Write protection, use the `+` symbol between directives on a single `#MODE` line.

Syntax

```
Run from Flash

#MODE RUNFLASH      // This prog intended to be 'front end' and run from FLASH

Save to Disk

#MODE SAVE_TO_DISK  // This prog will be saved to uSD card on DIABLO-

Multiple

// This program will reside on FlashBank 2 and will be Read/Write Protected
#MODE FLASHBANK_2 + FLASH_READ_PROTECT + FLASH_WRITE_PROTECT
```

#STACK

The Stack is a fixed portion of RAM on the system Heap (The Heap comprises all the RAM available to the user and is largely managed automatically) when a program starts. It contains local variables, parameters, and the return address of the previous function for the current function. The same information for all previous functions is also stored on the stack.

`#STACK xxx` is used to define the size of RAM, where `xxx` is the desired size. The default value, when not specified is **200** 'entries' in size, each entry is a word long.

In ViSi-Genie, the `#STACK` is handled automatically. In Designer and ViSi, the `#STACK` needs to be handled by the User, as the size of the Stack required depends on the code that has been written. Generally, the default is adequate, although for programs with recursive functions, functions with large quantities of variables, or where the function 'depth' is large, a larger value might be needed.

The compiler produces a `STACK_ESTIMATE` value in the `.aux` file, this value assumes every function calls every other function and there is no recursion, so it is only useful as a guide. After compiling, the `.aux` file can be opened in a text editor and the `STACK_ESTIMATE` value can be seen. Remember though, this is a guide only and may be insufficient for your application.

Example

```
#STACK 500
```

If the `#STACK` value is too small, this will result in an **EVE Stack Overflow** error. Too large, and you simply waste RAM and thus could result in your program running out and causing other errors or undesirable effects.

#inherit

`#inherit "filename"` includes another source file. Inherited files can be nested. Inherited files can now also contain `#DATA` , functions, and variables. There is no special ordering required for functions. **main** function can now be the first function in the program if required, however, pre-processor calculation cannot be forward referenced. There is now also a mechanism to allow selected portions of an inherited file to be compiled on demand (see `#USE` and `USING` below).

Syntax: `#inherit "filename.fnc"`

Example:

```
#platform "GOLDELOX"
#inherit "4DGL_16bitColours.fnc"

func main()
    // do something
endfunc
```

Expression and Operators

Operators make 4DGL a powerful language. An operator is a function which is applied to values to give a result. These operators take one or more values and perform a useful operation. The operators could be `+` , `-` , `&` etc.

Most common ones are arithmetic operators. Other operators are used for comparison of values, combination of logical states and manipulation of individual binary digits.

Expressions are a combination of operators and values. The values produced by these expressions can be used as part of even larger expressions or they can be stored in variables.

Assignment Operator

The assignment operator `:=` is commonly used to give/assign a value to variable.

Syntax:

Define a Variable

```
var variableName := constantValue;
```

Define an Array

```
var arrayName[arraySize] := [element1, element2, element3, /* ... */ elementN];
```

Update Variable

```
variableName := newConstantValue;  
variableName := equation;
```

Update Array Element

```
var arrayName[arraySize];  
arrayName[arrayIndex] := newConstantValue;  
arrayName[arrayIndex] := equation;
```

Address Modifier

[&] Get Variable Address

```
a := &myArray[0];  
a := myArray;      // same as above. The array name without indices  
a := &b;           // implies an address, same as in the C language
```

[*] Use Variable as Pointer

Example 1

```
a := *b;           // Can be used on the right side and/or  
*j := myArray;     // left side and/or  
*d := *s;          // even both sides of assignment operator.
```

Example 2

```
var a;             // a is declared as a variable  
var b;             // b is a variable  
var j[3] := [5, 200, 500]; // j is an array with initial values  
  
a := j;            // a holds the address of j  
b := a[0];         // b is equal to the first element of array j  
  
print("j[0]: ", b, "\n");
```

Note

When using a variable as a pointer, it is a good practice to declare it as such. Otherwise, the compiler will generate a notice. To illustrate, [Example 2](#) show a variable being used as a pointer, but not declared as such.

[Example 2](#) would compile and print **j[0]: 5** on the display module. However, the message area would display the notice below.

```
Notice: variable 'a' is being indexed (line 12 file:pomterTest2.4dg)  
0 errors  
0 warnings  
1 notice  
No Errors, code size = 84 bytes out of 14400 total  
Initial RAM size = 200 bytes out of 14400 total  
Program will run from ram so total initial RAM size = 284 bytes out of 14400 total  
Download to Flash successful.
```

Hence the correct way is to declare **"a"** explicitly as a pointer.

Example

```
var *a;            // a is declared as a pointer  
var b;             // b is a variable  
var j[3] := [5, 200, 500]; // j is an array with initial values  
  
a := j;            // a is a pointer which holds the address of j  
b := a[0];         // b is equal to the first element of array j  
  
print("j[0]: ", b, "\n");
```

The compiler notice is now removed.

```
0 errors  
0 warnings  
0 notices  
No Errors, code size = 84 bytes out of 14400 total  
Initial RAM size = 200 bytes out of 14400 total  
Program will run from ram so total initial RAM size = 284 bytes out of 14400 total
```

Arithmetic Operators

Arithmetic Operators are the most commonly used type of operator as these are used to perform basic mathematical operations for computing values such as from sensor readings, counting data etc.

There are five operators that fall under this category: **+**, **-**, *****, **/** and **%**

*****, **/** and **%** have the higher precedence and the operation will be performed before **+** or **-** in any expression.

Parenthesis **[(,)]** should be used to enforce a different order of evaluation. Where division is performed between two integers, the result will be an integer, with the remainder discarded. If a program is ever required to divide a number by zero, this will cause an error, usually causing the program to crash.

[+] Addition

```
val1 := 5;
val2 := 10;
sum := val1 + val2;
```

[-] Subtraction

```
val1 := 5;
val2 := 10;
diff := val2 - val1;
```

[*] Multiplication

```
time := 10;
velocity := 5;
displacement := velocity * time;
```

Note

The overflow (bits 16 to 31) can be read by the **OVF()** function.

[/] Division

```
delta := 5;
length := 10;
strain := delta / length;
```

Note

The remainder can be read by the **OVF()** function.

[%] Modulus

```
a := 3;
b := 11;
remainder := b % a; // remainder is 2
```

Comparison Operators

[==] Equality

```
if (index == count)
    // ...
    print("count complete");
endif
```

[!=] Inequality

```
if (denominator != 0)
    // ...
    result := numerator / denominator;
endif
```

[>] Greater Than

```
while (index > 0)
    // ...
    index--;
wend
```

[>=] Greater Than or Equals

```
while (index >= 0)
    // ...
    index--;
wend
```

[<] Less Than

```
while (index < 10)
    // ...
    index++;
wend
```

[<=] Less Than or Equals

```
while (index <= 10)
    // ...
    index++;
wend
```

Logical Operators

[!] Unary NOT

The unary NOT operator sets a 1 if all bits in a variable are zero, otherwise sets 0

Example 1

```
while (!x)
    // continue operation while x == 0
```

```
wend
```

The NOT (!) operator inverts the Boolean result of a value, or the result of an expression. For example, if a variable named flag is currently 55, prefixing the variable with a ! character will make the result FALSE (0).

Example 2

```
var flag := 55;      // variable is non zero
var flag2;
flag2 := !flag;      // flag2 set to FALSE (0)
flag2 := !flag2;     // now flag2 set to TRUE (1)
```

[~] Unary 2's Complement

The unary 2's Complement (~) operator inverts all bits.

```
x := 0x5555;
print([HEX]~x); // Prints 0xAAAA
```

[&&] Logical AND

```
if (x < 10 && x > 5) print("x within range");
```

[||] Logical OR

```
if (x > 10 || x < 5) print("x out of range");
```

[^] Logical XOR

```
if ((A < B) ^ (C < D))
    putstr("Expression is true");
endif
```

Bitwise Operators

[&] Bitwise AND

The Bitwise AND is represented by a single ampersand (&). It makes a bit-by-bit comparison of two numbers. Any corresponding position in the binary sequence of each number where both bits are 1 result in a 1 appearing in the same position of the resulting number. If either bit position contains a 0 then a zero appears in the result.

Example

```
var j := 0b000000010101011;
var k := 3;

var r;
r := k & j; // Perform bitwise AND on variables k and j

print("The result is ", r); // Result is 3
```

[|] Bitwise OR

The bitwise OR performs a bit-by-bit comparison of two binary numbers. The OR operator places a 1 in the result if there is a 1 in the first or second operand.

Example

```
var j := 0b000000010101011;
var k := 3;

var r;
r := k | j; // Perform bitwise OR on variables k and j

print("The result is ", r); // Result is 171
```

[^] Bitwise XOR

The bitwise XOR sets a 1 if one or other corresponding bit positions in the two numbers is 1. If both positions are a 1 or a 0 then the corresponding bit in the result is set to a 0.

Example

```
var j := 0b000000010101011;
var k := 3;

var r;
r := k ^ j; // Perform bitwise OR on variables k and j

print("The result is ", r); // Result is 168
```

[<<] Shift Left

The bitwise left shift moves each bit in a binary number a specified number of positions to the left. As the bits are shifted to the left, zeros are placed in the vacated rightmost (low order) positions. Note that once the left most (high order) bits are shifted beyond the size of the variable containing the value, those high bits can be read with `OVF()` function.

On GOLDELOX, `peekW(VM_OVERFLOW)` can also be used to read `OVF()`.

Example


```
var k := 0b0110111100000000; // 28416
var r;
pokeW(VM_OVERFLOW, 0); // clear the overflow register (GOLDELOX only)
r := k << 3; // shift k 3 bit positions to the left
print("The result is ", r, "OVF() = ", OVF());
```

The example code would display the shifted result, which is 3552 and the overflow value is 3.

GOLDELOX:

The `VM_OVERFLOW` register is not cleared prior to a shift, this allows you to do interesting things such as rotating an array, The `VM_OVERFLOW` register must be cleared using `pokeW(VM_OVERFLOW, 0)` (or pre-set to a required value) prior to using the shift instruction if you wish to obtain the correct result. The most significant bit goes out and into the `VM_OVERFLOW` register which can be read by `OVF()` function, or by using `peekW(VM_OVERFLOW)`.

PICASO/DIABLO-/PIXI-:

`OVF()` is cleared prior to shifting, there is no `VM_OVERFLOW` register on these processors.

[>>] Shift Right

A bitwise right shift is much the same as a left shift except that the shift takes place in the opposite direction.

Note that the low order bits that are shifted off to the right can be read with `OVF()`.

On GOLDELOX, `peekW(VM_OVERFLOW)` can also be used to read `OVF()`.

The vacated high order bit(s) position is replaced with zeros.

Example

```
var k := 0b0000000010101011; // 171
var r;
pokeW(VM_OVERFLOW, 0); // clear the overflow register (GOLDELOX only)
r := k >> 3; // shift k 3 bit positions to the right
print("The result is ", r, "OVF() = ", OVF());
```

The above code would display the shifted result, which is 21 and the overflow value is 16384.

GOLDELOX:

The `VM_OVERFLOW` register is not cleared prior to a shift, this allows you to do interesting things such as rotating an array. The `VM_OVERFLOW` register must be cleared using `pokeW(VM_OVERFLOW, 0)` (or pre-set to a required value) prior to using the shift instruction if you wish to obtain the correct result. The least significant bit goes out and into the `VM_OVERFLOW` register which can be read by `OVF()` function, or by using `peekW(VM_OVERFLOW)`.

PICASO/DIABLO-/PIXI-:

`OVF()` is cleared prior to shifting, there is no `VM_OVERFLOW` register on these processors.

Shorthand Notations

The `iterator(...)` function can be used to modify the post or pre increment value for the next execution of the Post/Pre increment and decrement operation. The increment value is automatically set back to 1 after the next Post/Pre increment and decrement operation.

[var++] Post-Increment

```
x := a * b++;
// equivalent to
// x := a * b;
// b := b + 1;
```

[++var] Pre-Increment

```
x := ++b * a;
// equivalent to
// b := b + 1;
// x := a * b;
```

[var--] Post-Decrement

```
x := a * b--;
// equivalent to
// x := a * b;
// b := b - 1;
```

[--var] Pre-Decrement

Simple

```
x := --b * a;
// equivalent to
// b := b - 1;
// x := a * b;
```

Using `iterator(...)`

```
// Using iterator ...
iterator(10);
myarray[x++] := 123;

// equivalent to
// myarray[x] := 123;
// x := x + 10;
```

Compound Assignment Operators

4DGL now provides several operators designed to combine an assignment with a mathematical or logical operation. These are primarily of use when performing an evaluation where the result is to be stored in one of the operands.

For example, you may write an expression as follows:

[+=] Operator

```
k += j;      // Add j to k and place result in k
```

[-=] Operator

```
k -= j;      // Subtract j from k and place result in k
```

[*=] Operator

```
k *= j;      // Multiply k by j and place result in k
```

Note

Any overflow situation from the math operators can be obtained using `OVF()`. On GOLDELOX `peekW(VM_OVERFLOW)` can also be used.

[/=] Operator

```
k /= j;      // Divide k by j and place result in k
```

Note

Remainder can be obtained using `OVF()`. On GOLDELOX `peekW(VM_OVERFLOW)` can also be used.

[%=] Operator

```
k %= j;      // Perform Modulo of j on k and place result in k
```

Compound Bitwise Operators

Like the arithmetic operators, each bitwise operator has a corresponding compound operator that allows the operation and assignment to be performed using a single operator.

Note

Any overflow situation from the math operators can be obtained using `OVF()`. On GOLDELOX `peekW(VM_OVERFLOW)` can also be used.

[&=] Compound Bitwise AND

```
k &= j;      // Perform a bitwise AND of k and j and assign result to k
```

[|=] Compound Bitwise OR

```
k |= j;      // Perform a bitwise OR of k and j and assign result to k
```

[^=] Compound Bitwise XOR

```
k ^= j;      // Perform a bitwise XOR of k and j and assign result to k
```

Ternary Operator

4DGL now includes the ternary operator, a shortcut way of making decisions.

Syntax: `[condition] ? [true expression] : [false expression]`

The way this usually works is that *[condition]* is replaced with an expression that will return either TRUE(1) or FALSE(0). If the result is true, then the expression that replaces the *[true expression]* is evaluated. Conversely, if the result was false then the *[false expression]* is evaluated.

Examples

```
var k := 20, j := 40;
var r;
r := (k > j) ? k : j ;
print("Larger number is ", r);
```

The above code example will evaluate whether k is greater than j, this will evaluate to false resulting in j being returned to r.

```
print((RAND() < 10000) ? 1 : 0);
```

The above code example will print a **1** if the random number is < 1000, else it will print a **0**.

```
print([STR]((RAND() < 10000) ? "low " : "high"));
```

The above code example will print **low** if the random number is < 1000, else it will print **high**.

```
(n >= 45 && n <= 55) ? myfunc1() : myfunc2();
```

The above code example will call `myfunc1()` if the number lies between 45 and 55 inclusive, else it will call `myfunc2()` . The functions may have arguments if required.

Any number of comma-separated expressions may occur to the left and right of the colon (`:`), however, only the right-most element will return a value. For example, in the following code:

```
r := (n >= 45 && n <= 55) ? myfunc1() : myfunc2(), x++;
```

`myfunc1()` will be called if the number lies between 45 and 55 inclusive and `r` will receive its return value, else `myfunc2()` will be called, but returns the value of `x` (*x is then incremented*).

The return value (if any) from `myfunc2()` is discarded.

Language Flow Control

if - else - endif

The **if-else** statement is a two-way decision statement. The **if** statement answers the question, "*Is this true or false?*", then proceeds on some action based on this. If the condition was **true**, then the statement(s) following the **if** is executed and if the condition was false then the statement(s) following the **else** is executed.

Syntax

Single Line if

```
if (condition) statement;
```

Single Line if-else

```
if (condition) statement; else statement;
```

Multi-line if-else-endif

```
if (condition)
    [statements]
else
    [statements]
endif
```

Description	
condition	Required conditional expression to evaluate
statement	Optional single statement
statements	Optional block of statements or single statement to be executed

Example

```
func collision()
    if (ball_x <= LEFTWALL)
        ball_x := LEFTWALL;
        ball_colour := LEFTCOLOUR;
        xdir := -xdir;
    endif

    if (ball_x >= RIGHTWALL)
        ball_x := RIGHTWALL;
        ball_colour := RIGHTCOLOUR;
        xdir := -xdir;
    endif
endfunc
```

while - wend

Loop through a block of statements while a specified condition is true. Note that the **while** statement may be used on a single line without the **wend** statement. The **while-wend** loop evaluates an expression before executing the code up to the end of the block that is marked with a **wend**. If the expression evaluates to FALSE on the first check, then the code is not executed.

Syntax

Single Line while

```
while (condition) statement;
```

Blocking while Loop

```
while (condition);
// Program will stop here, continuously evaluating the
// condition and proceeds when condition is FALSE
```

Multi-line while-wend

```
while (condition)
    [statements]
    [break;]
    [continue;]
wend
```

	Description
condition	Required condition to evaluate each time through the loop. The loop will be executed while the condition is true.
statement	Optional single statement to be executed
statements	Optional block of statements or single statement to be executed
<i>Related Statements</i>	<i>only applicable in block (multi-line) mode</i>
<code>break;</code>	Break out of the while loop by jumping to the statement following the <code>wend</code> keyword. (optional)
<code>continue;</code>	Skip back to beginning of the while loop and re-evaluate the condition. (optional)

Examples:

Single Line *while*

```
i := 0;
val := 5;

while (i < val) myVar := myVar * i++;
```

Multi-line *while-wend*

```
i := 0;
val := 5;

while (i < val)
    myVar := myVar * i;
    i++;
wend
```

Nested *while* Loops

```
// This example shows the nesting of the while..wend loops
var rad, color, counter;
func main()
    color := 0xF0F0;
    gfx_Set(0, 1);    // set PenSize to 1 for outline objects
    while (counter++ != 1000)
        rad := 10;
        while (rad < 100)
            gfx_Circle(120, 160, rad++, color++);
            gfx_Ellipse(120, 160, rad++, 20, color++);
            gfx_Line(120, 160, 20, rad++, color++);
            gfx_Rectangle(10, 10, rad++, rad++, color++);
        wend
    wend
endfunc
```

repeat - until / forever

Loop through a block of statements until a specified condition is **true**. The statement block will always execute at least once even if the `until(condition)` result is **true**. For example, you may need to step through an array until a specific item is found.

The **repeat** statement may also be used on a single line with `until(condition);`

Syntax

Single Line *repeat-until*

```
repeat (statement); until(condition);
```

Multi-line *while-wend*

```
while (condition);
// Program will stop here, continuously evaluating the
// condition and proceeds when condition is FALSE
```

Multi-line *while-wend*

```
while (condition)
    [statements]
    [break;]
    [continue;]
wend
```

	Description
statement	Optional single statement to be executed
statements	Optional block of statements or single statement to be executed
forever	Required keyword to specify the end of the repeat loop block if not using <code>until(condition);</code>
until	Required keyword to specify the end of the repeat loop block if not using <code>forever</code>
condition	Required condition to evaluate at the end of loop. The loop will be repeated until the condition is true.
<i>Related Statements</i>	<i>only applicable in block (multi-line) mode</i>

	Description
<code>break;</code>	Break out of the repeat loop by jumping to the statement following the <code>until(condition)</code> or <code>forever</code> keywords. (optional)
<code>continue;</code>	Skip back to beginning of the repeat loop (optional)

Examples:

Single Line *repeat-until*

```
i := 0;
repeat i++; until(i >= 5);
```

Multi-line *repeat-until*

```
i := 0;
repeat
  myVar := myVar * i;
  i++;
until (i >= 5);
```

Multi-line *repeat-forever*

```
i := 0;
repeat
  myVar := myVar * i;
  i++;
  if (i >= 5) break;
forever
```

goto

The goto instruction will force a jump to the label and continue execution from the statement following the label. Unlike the `gosub` there is no return. It is strongly recommended that `goto` **SHOULD NEVER BE USED**, however, it is included in the language for completeness.

The `goto` statement can only be used within a function and all labels are private within the function body. All labels must be followed by a colon `:`.

Syntax

```
goto label; // branches to the statements at label:
// ...
// insert code here
// ...

label:
[statements]
```

	Description
label	Required label for the goto jump

Example

```
func main()

  if (x < 20) goto bypass1;
  print("X too small");
  x := 20;

bypass1:
  if (y < 50) goto bypass2;
  print("Y too small");
  y := 50;

bypass2:
  // more code here

endfunc
```

for - next

Repeats a set of statement certain number of times.

The `for` statement contain 3 main sections: initialisation, condition and update

These three sections are separated by semicolon `;`. The for-loop is typically started with:

```
for (initialisation; condition; update)
```

It is possible to do things like `x++`, `x := x + 10`, or even `x := random(5)` as a post operation/update. The update section can be used to call other functions that do nothing to the variable checked in the condition section but still have a useful effect on the code.

Every single one of the sections may be empty, though the semicolons still must be there. If the condition is empty, it is evaluated as **true** and the loop will repeat until something else stops it, like a `break` statement within the loop body.

If a for loop is on a single line, `next` may be omitted.

Syntax

Single Line *for-next*

```
for (initialisation; condition; update) [statement];
```

Multi-line *for-next*

```
for (initialisation; condition; update)
    // statements to execute while the condition is true
    [statements]
next
```

Description	
initialisation	Assign a value to an already existing variable. More than one variable may be initialized here, not necessarily associated with the loop control.
condition	Tells the program that while the conditional expression is true the loop should continue to repeat itself.
update	A statement that may update the variable being used in the conditional expression or call a function
statement	Optional single statement to be executed
statements	Optional block of statements or single statement to be executed
Related Statements	<i>only applicable in block (multi-line) mode</i>
<code>break;</code>	Break out of the for-next loop by jumping to the statement following the <code>next</code> . (optional)
<code>continue;</code>	Skip back to beginning of the for/next updater (last section). (optional)

Example

Single Line *for-next*

```
for (n := 5; n < 15; n++) myfunc();
```

Multi-line *for-next*

```
func main()
    for (rad:=0; rad < 100; rad++)
        gfx_Circle(63, 63, rad, color);
    next
endfunc
```

switch - endswitch

Runs one of several groups of statements, depending on the value of an expression. In 4DGL, there are 2 different classes of **switch-case** statement.

Switch with Expression

The first type is **switch with expression**. It evaluates the conditional expression following the **switch** statement and tests it against numerous constant values (the cases). A case that matches the value of the expression executes the statement block.

Syntax

```
switch (expression)
    case constant1:
    case constantN:      // see note #1
        [statements];   // statementsBlock1
        break;          // optional break

    case constant2:
        [statements];   // statementsBlock2A
        continue;       // optional continue
        [statements];   // statementsBlock2B
        break;          // optional break

    case constant3:
        [statements];   // statementsBlock3
        break;          // optional break

    default:
        [statements];   // defaultStatementBlock
        break;          // optional break
endswitch
```

The **case** statements and the **default** statement can occur in any order in the **switch** body. The default statement is optional and is activated if none of the constants in the case statements can be matched. The **continue** statement may be used to force the **switch** statement to re-evaluate the expression and restart the **switch** body.

Description	
expression	Expression to check the value of for each of the constants
constantN	Values to check the expression with
statements	Optional block of statements or single statement to be executed
<code>break;</code>	Break out of the switch-endswitch by jumping to the statement following the <code>endswitch</code> . (optional)
<code>continue</code> <code>;</code>	Force the switch statement to re-evaluate the expression and restart the switch body. (optional)

Note

- Multiple case statement values can be associated with a statement block.
- The compiler produces the most efficient code if the case values are a consecutive range of numbers in which case it can produce a simple linear vector table which will not only be compact but will execute a bit faster than non-consecutive values. This does not mean that the cases need to be in this order, they are sorted during compilation to produce a linear table if possible. If the case values are not ordered, the compiler still determines the best strategy for building case statements depending on case values, eg. if you use values less than 255 it can build a match table for the vectors in a linear byte array (similar to the `lookup8` function), but if the values exceed 255 it needs to build a match table with words (similar to the `lookup16` function).

Example

```
var msg[100];
func main()
  var result;
  to (msg); putstr("This msg contains 4DGL and other words");
  print("string has ", strlen(msg), " characters\n");
  result := test1();
  print("found '4DGL' at position ", result, "\n");
  repeat forever
endfunc

// Uses four nested switch statements in a loop to scan character patterns.
// Not a good way to scan for patterns, but a good switch test.
// Note, the breaks have been commented out as they are redundant as there are no more cases
// in the nested switch level.

func test1()
  var i, p;
  p := str_Ptr(msg);
  for (i := 0; i < strlen(msg) - 3; i++)
    switch (str_GetByte(p + i))
      case '4':
        switch (str_GetByte(p + i + 1))
          case 'd':
            case 'D':
              switch (str_GetByte(p + i + 2))
                case 'g':
                case 'G':
                  switch (str_GetByte(p + i + 3))
                    case 'l':
                    case 'L':
                      return i;
                  endswitch
                break;
              endswitch
            break;
          endswitch
        break;
      endswitch
    break;
  next
  return 0;
endfunc
```

Switch without Expression

The second type of switch statement in the **expressionless switch** which allows a complete expression to be evaluated for each **case**. It can be thought of as a sequence of **if-endif** blocks, with the optional **break** acting as a **goto** to skip the rest of the evaluations or **continue** to act as a **goto** to restart the switch block from the top. Overall, this type of switch usually gives a neater appearance than the **if-else-endif** construction and is easier to visualize.

Syntax

```
switch
  case (expression1) // see note #2
    [statements]; // statementsBlock1
    break; // optional break

  case (expression2)
    [statements]; // statementsBlock2A
    continue; // optional continue
    [statements]; // statementsBlock2B
    break; // optional break

  case (expression3)
    [statements]; // statementsBlock3
    break; // optional break

  case (expressionN)
    [statements]; // statementsBlockN
    break; // optional break

  default:
    [statements]; // defaultStatementBlock
    break; // optional break
endswitch
```

Description	
expressionN	Expressions/Condition to check
statements	Optional block of statements or single statement to be executed
<code>break;</code>	Break out of the switch-endswitch by jumping to the statement following the <code>endswitch</code> . (optional)

Description	
<code>continue;</code>	Force the switch statement to re-evaluate the expression and restart the switch body. (optional)

 **Note**

1. For this class of switch, the switch statement has no expression.

2. Each case must be a parenthesised comparison expression, and no colon follows.

Break and Continue

It is possible exit from a **while**, **repeat**, or **for** loop at any time by using the **break** statement. When the execution path encounters a break statement the looping will stop, and execution will proceed to the code immediately following the loop.

It is important to note that in the case of nested loops the **break** statement only exits the current loop leaving the outer loop to continue execution.

The **continue** statement causes all remaining code statements in a loop to be skipped and execution to be returned to the top of the loop.

```
func main()
  while (n < 20)
    n++;
    if (n == 3) continue;
    if (n == 7) break; // output is 12456
    print(n);
  wend
endfunc
```

In the above example, **continue** statement will cause the printing of **n** be skipped when it equals 3 and the **break** statement will cause loop termination when **n** reaches 7.

Functions and Subroutines

func - endfunc

A function in 4DGL, just as in the C language, is a block of code that performs a specific task. Each function has a unique name, and it is reusable i.e., it can be called and executed from as many different parts in a 4DGL program. A function can also optionally return a value to the calling program. Functions can also be viewed as small programs on their own.

Some of the properties of functions in 4DGL are:

- A function must have a unique name and it is this name that is used to call the function from other functions, including `main()`.
- A function performs a specific task, and the task is some distinct work that the program must perform as part of its overall operation.
- A function is an independent smaller program which can be easily removed and debugged.
- A function will always return to the calling program and can optionally return a value.

Functions have several advantages:

- Less code duplication –easier to read / update programs.
- Simplifies debugging –each function can be verified separately.
- Reusable code –the same functions can be used in different programs.

 **Note**

- Functions must be created before they can be used.
- Functions can optionally return a value

Syntax

Return Value	
	<pre>func name([var parameter1, var parameter2, /* ... */ var parameterN]) [statements] [return value;] endfunc</pre>
No Return Value	
	<pre>func name([var parameter1, var parameter2, /* ... */ var parameterN]) [statements] [return;] endfunc</pre>

Description	
name	Required name/identifier for the function
parameterN	Optional list of parameters to be passed to the function
statements	A block of statements that make up the body of the function
return	Exit this function, returning control back to the calling function (optional)

Description	
return value	Exit this function, with a variable or expression to return a value for this function (optional)

Example

Passing Parameters

```
/*
... This example shows how to create a function and its usage in
... calling and passing parameters.
*/

func add2(var x, var y)
    var z;
    z := x + y;
    return z;
endfunc

func main()
    var a;
    a := add2(10, 4);
    print(a);
endfunc
```

Single line Function

```
func myfunc(var n) print("Error: ", [STR] errorstrings[n]); endfunc
```

As long as there are no forward references that need to be calculated, the ordering of functions no longer needs to be ordered with strict backward referencing.

A 4DGL program must have a starting origin where the point of execution begins. When a 4DGL program is launched, EVE processor takes control and needs a specific starting point. This starting point is the **main()** function and every 4DGL program must have one. The **main()** function is the block of code that makes the whole program work.

Arguments and Return Value

In other languages and in mathematics a function is understood to be something which produces a value or a number. That is, the whole function is thought of as having a value. In 4DGL it is possible to choose whether a function will have a value. It is possible to make a function return a value to the place at which it was called.

Example

```
bill = CalculateBill(data, ...);
```

The variable bill is assigned to a function CalculateBill() and data are some data which are passed to the function. This statement makes it look as though CalculateBill() is a number. When this statement is executed in a program, control will be passed to the function CalculateBill() and, when it is done, this function will then hand control back. The value of the function is assigned to "bill" and the program continues. Functions which work in this way are said to return a value.

In 4DGL, returning a value is a simple matter. Consider the function CalculateBill() from the statement above:

```
func CalculateBill(var starter, var main, var dessert)
    // Adds up values
    var total;
    total := starter + main + dessert;
    return total;
endfunc
```

As soon as the return statement is met CalculateBill() stops executing and assigns the value total to the function. If there were no return statement, the program could not know which value it should associate with the name CalculateBill and so it would not be meaningful to speak of the function as having one value. Forgetting a return statement can ruin a program, then the value bill would just be garbage (no predictable value), presuming that the compiler allowed this to be written at all.

gosub - endsub

The gosub starts executing the statements at the label(subroutine name) until it reaches endsub and returns to continue after the gosub statement.

Note

- The gosub statement can only be used **within a function**.
- All gosub labels are private within the function body.
- All subroutines must end with endsub;
- All labels must be followed by a colon :
- ** A common mistake is to forget the 'return' to return from a subroutine within a function.

Syntax

```
gosub label;

label:
    [statements]
endsub;
```


The above syntax executes the statements at `label: .` When the `endsub;` statement is reached, execution resumes with the statement following the `gosub` statement. The code between `label:` and the `endsub;` statement is called a **subroutine**.

Example

```
func myfunc()
    gosub mysub1;
    gosub mysub2;
    gosub mysub3;
    print("\nAll Done\n")
    return; // return from function *** see below

mysub1:
    print("\nexecuted sub #1");
endsub; // return from subroutine

mysub2:
    print("\nexecuted sub #2");
endsub; // return from subroutine

mysub3:
    print("\nexecuted sub #3");
endsub; // return from subroutine
endfunc

func main()
    myfunc();
endfunc
```

Indexed gosub - endsub

It is possible to use a list of **gosub** labels to search from using the specified **index**. Execution resumes with the statement following the **gosub** statement for index provided.

For example, if **index** is zero, the subroutine named by the first label in the list is executed. If **index** is one, then the second label and so on.

Note

- The indexed `gosub` can only be used **within a function**.
- If **index** is zero or greater than the number of labels, the first **label** is always executed.

```
gosub (index), (label1, label2, /* ... */ labelN);

label1:
    [statements]
endsub;

label2:
    [statements]
endsub;

labelN:
    [statements]
endsub;
```

Example

```
func myfunc(var key)

    var r;
    to (COM0); // set redirection for the next print command
               // to the COM port

    r := lookup8(key, "fbsclx?h");
    gosub(r), (unknown, forward, backward, set, clear, load, exit, help);

    goto done;

help:
    putstr("Menu f,b,i,d,s,c,l or x (? or h for help)\n");
endsub;

unknown:
    print("\nBad command '", [CHR]key, "' ?");
    /* more code here */
endsub;

forward:
    print("\nFORWARD ");
    /* more code here */
endsub;

backward:
    print("\nBACKWARD ");
    /* more code here */
endsub;

set:
    print("\nSET ");
    /* more code here */
endsub;

clear:
    print("\nCLEAR ");
    /* more code here */
endsub;

load:
    print("\nLOAD ");
    /* more code here */
endsub;

exit:
```

```
        print("\nEXIT      ");
        print("\nbye....");
        /* more code here */
        r := -1;    // signal an exit
    endsub;

done:
    return r;

endfunc

func main()
    var ch;
    putstr("Open the Workshop4 terminal\n");
    putstr("Enter f,b,i,d,s,c,l or x\n");
    putstr("Enter ? or h for help\n");
    putstr("Enter x for exit\n");
    ch := '?';

    goto here; // enter here to show help menu first up

    repeat
        while ((ch := serin()) < 0); // wait for a character

here:
        if (myfunc(ch) == -1) break;

        // keep going until we get the exit command
        forever

        putstr("\nEXITING");
    endfunc
```

SystemReset

This function resets and restarts the program, it is the equivalent of a 'cold boot' (i.e., a total hardware reset). There is a 2 second delay before the program restarts, this is due to the EVE boot procedure time.

Syntax: SystemReset();

Example

```
if (resetNow)
    SystemReset();
endif
```

ProgramExit

This function resets contrast to 0 and puts the display into low power sleep mode. For some devices, the only wakeup procedure is a reset or power cycle. Refer to individual module or chip specifications for information on other sleep/wakeup modes.

Syntax: ProgramExit();

Example

```
if (exitNow)
    ProgramExit();
endif
```

argcount

This compiler function returns then number of arguments required by a PmmC or user function. It is used often for getting the argument count when using the @ operator when using a function pointer.

Syntax: argcount(functionName);

Argument	Description
functionName	Specifies the User/PmmC function to check for number of arguments

Example

```
func aIsLarger(var a, var b)
    return a > b;
endfunc

func main()
    print("argcount :=", argcount(aIsLarger));
endfunc
```

[@] Argument Pointer

Function arguments can now be passed using the special pointer operator @ .

Example

```
#constant rsize argcount(gfx_Rectangle)
// define rsize for number of args to gfx_Rectangle()

var rect[rsize * 4], n;
// an array to hold info for 4 rectangles

func main()
    // initialize some default rectangle co-ords
    *rect := [10, 10, 40, 40, RED, 88, 10, 118, 40, GREEN, 10, 88, 40, 118, BLUE, 88, 88, 118, 118, YELLOW];

    for (n := 0; n < 4 * rsize ; n += rsize)
        gfx_Rectangle(@ rect+n);
```

```

    // draw all rectangles using arg pointer offset by n
next
repeat forever // done
endfunc

// alternatively, the ++ iterator can be employed which is a little
// more code efficient, a little faster in execution speed and allows
// a little more flexibility. Note that the iterator value is not
// 'sticky' and is reset to 1 once 'ndx++' is executed.

ndx := 0;
while (ndx < 4)
    iterator(rsize);
    // set the iterator to the size of args for 'ndx++'.....
    gfx_Rectangle(@rect + ndx++);
    // draw new rectangles, bump iterator
wend
```

Revision History

Revision	Date	Content
5.1	23/11/2012	Reformatted, minor document updates
5.2	27/02/2013	Fixed Case example which has : marks in error
5.3	24/07/2013	Added some detail for DIABLO16 Processor
6.0	01/05/2017	Updated formatting and contents
6.1	11/04/2019	Reformatted, minor document updates, cosmetic changes
6.2	21/11/2019	Removed duplicate chapters and fixed formatting
6.3	26/08/2020	Adding missing information to #MODE and added #STACK section
6.4	09/12/2020	Minor update to clarify OVF() usage for all processors, and that the VM_OVERFLOW register is being for GOLDELOX only. Precedence statement placed into its own heading.
6.5	13/05/2022	Improvement to wording in Section 4.2 Private Variables to fix a contradictory statement
6.6	05/07/2022	Correction on Switch without expression
6.7	06/02/2023	Modified for web-based documentation